

LiDAR 기반 지형 고도맵 복원

데이터 규약 규명 · 복원 도구 구축 · 실데이터 검증

작성일: 2026-08-05

사격통제팀

0. 요약

시뮬레이터의 Save LiDAR Data 기능으로 저장되는 CSV 를 분석해 지형 고도맵을 복원하는 방법과 도구를 확립했다.

항목	결과
데이터 규약	완전 규명 (수직각 부호 포함)
센서 원점	점군에서 역산 가능, 잔차 0.0001 m
파일당 커버리지	반경 최대 149 m, 유효점 1,500여 개
저장 주기	약 0.2초
갱신 여부	정상 동작 확인 (이전 판단 정정)
복원 도구	<code>lidar_terrain.py</code>
실데이터 검증	21개 파일로 커버리지 23.5% (보간 후)

주요 정정 두 건

- 수직각 부호가 이전 분석과 반대다 (양수가 하향)
- LiDAR 미갱신 판단은 오진이었다 (전차가 정지해 있었을 뿐)

1. 데이터 구조

1.1 파일 형식

파일명 LidarData_t{초}_{소수}.csv 예: LidarData_t0030_48.csv → t = 30.48
행 수 2,880 = 방위 360 × 수직 8채널

열	내용
<code>angle</code>	방위 인덱스 0 ~ 359 (1도 간격)

열	내용
vertical_angle	수직각 8종
distance	광선 도달 거리
x, y, z	착탄점 월드 좌표
isDetected	탐지 여부

1.2 수직각 규약 — 이전 분석 정정

수직각 8종은 다음과 같다.

-22.50, -16.07, -9.64, -3.21, +3.21, +9.64, +16.07, +22.50

양수가 하향이다. 센서 원점을 역산해 실제 각도를 계산한 결과.

표기 수직각	원점 기준 실제 하강각
+3.21°	+4.66°
+9.64°	+9.64°
+16.07°	+16.07°
+22.50°	+22.49°

이전 분석에서는 음수를 하향으로 가정했으나 반대였다.

다만 "8채널 중 4개만 지면을 탐지한다"는 결론은 동일하다.

1.3 탐지 분포

수직각	탐지 수	비고
-22.50 ~ -3.21	0	상향 — 하늘을 향해 아무것도 맞지 않음
+3.21	58 ~ 다수	가장 완만 — 최대 거리에 제한됨
+9.64	360	
+16.07	360	
+22.50	360	가장 가파름 — 근거리

탐지율이 약 40% 인 것은 상향 4채널이 전부 미탐지이기 때문이다.

1.4 지면 도달 거리

센서 지상고를 h 라 하면 각 채널이 평지에 닿는 거리는

$$\text{도달거리} = h / \tan(\text{하강각})$$

지상고 2.36 m 인 실측 사례.

하강각	실측 평균	이론값
+3.21°	25.55 m	42.10 m (최대 거리에 잘림)
+9.64°	14.67 m	13.91 m
+16.07°	8.50 m	8.21 m
+22.50°	5.90 m	5.71 m

가장 완만한 +3.21° 채널이 유효 거리를 결정한다.

$$\text{최대 지면 도달거리} = \text{지상고} \times 17.8$$

지상고 2.36 m → 42 m

Max Distance 설정으로 추가 제한된다(실측 사례에서 30 m, 150 m 두 가지 확인).

2. 센서 원점 역산

2.1 원리

파일에는 센서 원점이 기록되지 않는다. 그러나 각 점에 대해

$$|\text{점} - \text{원점}| = \text{distance}$$

가 성립하므로, 이를 만족하는 원점을 최소제곱으로 구할 수 있다.

1,500여 개 점의 **과잉결정 문제**이므로 정확도가 매우 높다.

2.2 결과

잔차 RMS 0.0000 m
잔차 최대 0.0001 m

`/info` 의 `lidarOrigin` 필드가 없어도 원점을 정확히 알 수 있다.

2.3 구현

가우스-뉴턴 반복으로 구현했다. scipy 없이 numpy 만으로 동작한다.

0 ← 초기 추정
반복:

```
r = |P - o|
잔차 f = r - D
야코비안 J = -(P - o) / r
o ← o + lstsq(J, -f)
```

3. 갱신 여부 판정 — 이전 판단 정정

3.1 초기 관측

파일 여러 개의 원점을 비교한 결과, $t = 7.03 \sim 36.55$ 구간(약 30초)에서 원점이 **(59.36, 27.58)** 에 고정되어 있었다.

- 포탑도 회전하지 않음 (`angle=0` 광선의 월드 방위가 항상 359.99°)
- 점군 차이가 **0.0001 m 미만** — 부동소수점 오차 수준

이를 갱신 이상으로 판단했다.

3.2 추가 데이터로 확인된 사실

이후 확보한 파일에서 원점이 명확히 이동했다.

```
t = 30.06   (30.71, 26.41)
t = 30.28   (31.62, 26.33) ← 0.91 m 이동
t = 30.48   (32.65, 26.20) ← 1.05 m 이동
```

0.2초에 약 1 m, 속도로 환산하면 약 4.5 m/s.

점군 해시도 모두 다르다.

3.3 결론

LiDAR 저장은 정상 동작한다.

원점이 고정되어 보였던 것은 전차가 실제로 움직이지 않았기 때문이다.

원점 (59.36, 27.58) 은 시뮬레이터의 기본 스폰 지점이다.

3.4 파급 — `/info` 판단도 재검증 필요

이전에 `/info` 의 `lidarPoints` 가 갱신되지 않는다고 판단한 근거는 "`lidarOrigin` 이 1,171회 연속 동일"이었다.

그러나 당시 전차가 실제로 주행했는지 확인하지 않았다.

방금 확인한 것과 동일한 상황(정지 상태)이었을 가능성이 있다.

파일 방식과 `/info` 는 같은 스캔 결과를 쓸 것으로 보이므로,
파일이 정상이면 `/info` 도 정상일 개연성이 높다.

재검증 방법

1. 목적지를 지정해 전차를 확실히 주행시킨다
2. `/status` 로 위치가 실제로 변하는지 먼저 확인한다
3. 그 상태에서 `lidarOrigin` 이 따라오는지 관찰한다

4. 복원 도구 — `lidar_terrain.py`

4.1 기능

기능	내용
원점 역산	가우스-뉴턴 (numpy 만 사용)
격자 누적	폴더 내 전체 CSV 를 읽어 고도 격자에 누적
지면 추출	같은 셀은 최저값 채택
물체 후보	셀별 최고-최저 높이차
보간	IDW, 탐색 반경 제한
오차 추정	블록 단위 홀드아웃
진단	<code>--inspect</code> 로 원점 이동 여부 확인
연계	<code>to_terrain_memory()</code> 로 기존 모듈에 이식

4.2 설계상의 판단

같은 셀에 여러 점이 있으면 최저값을 쓴다.

평균을 쓰면 전차나 바위에 맞은 점이 지면 높이를 끌어올린다.

지형을 원하므로 최저값이 맞다.

셀별 최고-최저 차이를 따로 저장한다.

값이 크면 그 자리에 물체가 서 있다는 뜻이므로 장애물 탐지에 쓸 수 있다.

보간 오차는 블록 단위로 검증한다.

이 부분이 방법론적으로 중요하다(4.3절).

4.3 보간 오차 추정의 함정

무작위 셀 홀드아웃은 오차를 크게 과소평가한다.

같은 주행선 위의 이웃 점이 1 m 남짓 거리에 남아 있어

가린 셀을 쉽게 복원해버리기 때문이다.

실제 오차는 **주행선 사이에서** 발생하는데 그것을 검증하지 못한다.

수치 실험 결과.

주행 간격	무작위 홀드아웃 추정	실제 오차	배율
8 m	0.10 m	0.30 m	3배
20 m	0.09 m	0.86 m	10배
40 m	0.10 m	1.82 m	18배

따라서 **블록 단위로 가려야** 한다.

연속된 영역을 통째로 홀드아웃하면 실제 보간 상황에 가까워진다.

4.4 사용법

```
# 진단 - 원점이 이동하는지 확인
python lidar_terrain.py "폴더" --inspect

# 고도맵 복원
python lidar_terrain.py "폴더" --cell 2.0 --out heightmap
```

출력

```
heightmap_raw.npy      관측 격자 (미관측은 NaN)
heightmap_filled.npy   IDW 보간 격자
heightmap_count.npy    셀별 점 개수
heightmap_object.npy   셀별 최고-최저 (물체 후보)
heightmap_meta.txt     메타데이터
```

4.5 기존 모듈 연계

```
from lidar_terrain import LidarTerrain
lt = LidarTerrain(cell=2.0)
lt.load_dir("LidarData폴더")
tm = lt.to_terrain_memory()      # viewshed, 탄도 차폐 판정에 사용
```

`terrain.py` 의 `TerrainMemory` 로 변환되므로

`threat.py` 의 위협 지도 계산과 사격 붓의 탄도 차폐 판정에 바로 쓸 수 있다.

5. 실데이터 검증

5.1 조건

확보한 파일 21개(서로 다른 시각·위치)로 시험했다.

5.2 결과

스캔 21개 → 점 29,542개
관측 커버리지 4.1%
IDW 보간 후 23.5%
고도 범위 6.86 ~ 18.86 m
보간 오차 RMS 0.635 m MAE 0.379 m 최대 3.295 m
(8 m 블록 홀드아웃, n=223)

스캔 원점 범위는 x 30.7 ~ 168.0, z 26.2 ~ 279.7 이었다.

5.3 해석

파일 하나가 반경 최대 149 m 를 덮으므로 21개만으로도 맵 상당 부분이 채워진다.

0.2초마다 저장되므로 몇 분만 주행하면 수백 개가 쌓이고
맵 전체를 덮을 수 있다.

6. 대안과의 비교 — 주행 궤적 샘플링

LiDAR 를 쓸 수 없는 경우를 대비해, 전차 위치(`playerPos.y`)만으로
지형을 복원하는 방법의 소요를 계산했다.

6.1 필요한 주행량

시뮬레이터와 유사한 지형으로 수치 실험한 결과.

주행 간격	주행 거리	소요 시간	높이 RMS	가시선 판정 일치율
8 m	8.6 km	18분	0.38 m	97.0%
12 m	5.7 km	12분	0.43 m	96.8%
20 m	3.4 km	7분	0.76 m	90.9%
30 m	2.3 km	5분	1.20 m	85.0%
40 m	1.8 km	4분	1.63 m	76.1%

12 m 간격에서 효율이 꺾인다. 그보다 촘촘히 해도 개선이 거의 없고,
20 m 로 넓히면 급격히 나빠진다.

높이 오차보다 가시선 판정 일치율이 실질적 지표다.

지형 데이터의 용도가 *viewshed* 와 탄도 차폐 판정이기 때문이다.

6.2 주행 샘플링의 원리적 한계

차체 길이가 7.5 m 이므로 전차는 궤도 아래 지형의 평균 높이에 앉는다.

파장 7.5 m 미만의 기복은 아무리 촘촘히 주행해도 복원되지 않는다.

보간 오차가 아니라 관측 자체의 저역통과 필터다.

또한 전차가 갈 수 없는 곳(가파른 절벽, 물, 장애물 안쪽)은 영원히 알 수 없다.
경로 계획에는 문제가 없으나, **viewshed**에서는 그 지형이 시야를 막는지
판단할 수 없어 문제가 된다.

6.3 비교

항목	주행 궤적	LiDAR 파일
1회당 수집	점 1개	1,500여 점
커버 범위	현재 위치	반경 최대 149 m
저역통과	차체 7.5 m 평활화	없음
접근 불가 지역	불가	가능
전체 복원 소요	12분 이상 주행	수 분

LiDAR 방식이 모든 면에서 우월하다.

주행 샘플링은 LiDAR 를 쓸 수 없을 때의 대안으로만 의미가 있다.

7. 보조 수집원

주행과 LiDAR 외에도 지형 고도를 얻을 수 있는 경로가 있다.

수집원	내용	특징
착탄점	<code>/update_bullet</code> 의 (x, y, z)	전차가 갈 수 없는 곳도 샘플링 가능
적 전차 위치	<code>/info</code> 의 <code>enemyPos</code>	커버리지 2배
배치 오브젝트	<code>.map</code> 파일의 y 값	사전에 확보 가능

착탄점이 특히 유용하다. 사거리 130 m 안에서 360° 어디든 쏠 수 있으므로
절벽 아래나 물 건너편처럼 주행으로는 닿지 않는 지점을 찍을 수 있다.
다만 재장전 6.9초로 분당 9점이 한계다.

8. 활용 방향

8.1 즉시 적용 가능

용도	내용
탄도 차폐 판정	포물선이 중간 지형에 막히는지 검사
고도차 보정	표적이 높거나 낮을 때 조준각 조정

용도	내용
교전 포락 계산	고도차에 따른 최소·최대 사거리

8.2 위협 지도 (threat.py)

충	필요한 지형 정보
가시선	능선이 시야를 막는가
교전 포락	고도차 보정된 사거리
양각 사각지대	고도차가 도달 한계를 넘는가
선제 우위	(지형 무관)

8.3 개인 연구 (식생 은폐)

지형 고도는 **은폐(지형 기복)** 계산의 기반이며,
식생 은폐 효과와 분리해서 측정하려면 반드시 필요하다.

9. 남은 과제

항목	내용
<code>/info</code> LiDAR 재검증	확실히 주행시키며 <code>lidarOrigin</code> 추적
실시간 적용 방식 결정	<code>/info</code> 직접 수신 vs 파일 감시
파일 용량 관리	파일당 약 146 KB, 초당 5개 → 시간당 약 2.6 GB
커버리지 목표 설정	어느 정도 덮어야 실용적인지
센서 높이 조정	Y Pos 를 올리면 도달거리가 비례 증가 (× 17.8)

부록 — 관련 파일

파일	역할
<code>lidar_terrain.py</code>	LiDAR 파일 → 지형 고도맵
<code>terrain.py</code>	지형 기억, 가시선, 탄도 차폐 판정
<code>threat.py</code>	선제 사격 우위, 위협 지도

파일	역할
tank_server.py	주행 제어 + LiDAR 갱신 진단